

Exercise 1 report

1. Basic Information about the Platform

I was using two computers while doing this assignment, Windows 10 desktop (should maybe update to 11 some time :D) and a laptop running Linux Mint 22. I used the desktop primary for developing the services and run the services without docker first and then primary used the linux laptop for making the compose, testing the containerised application and developing while I was at the campus.

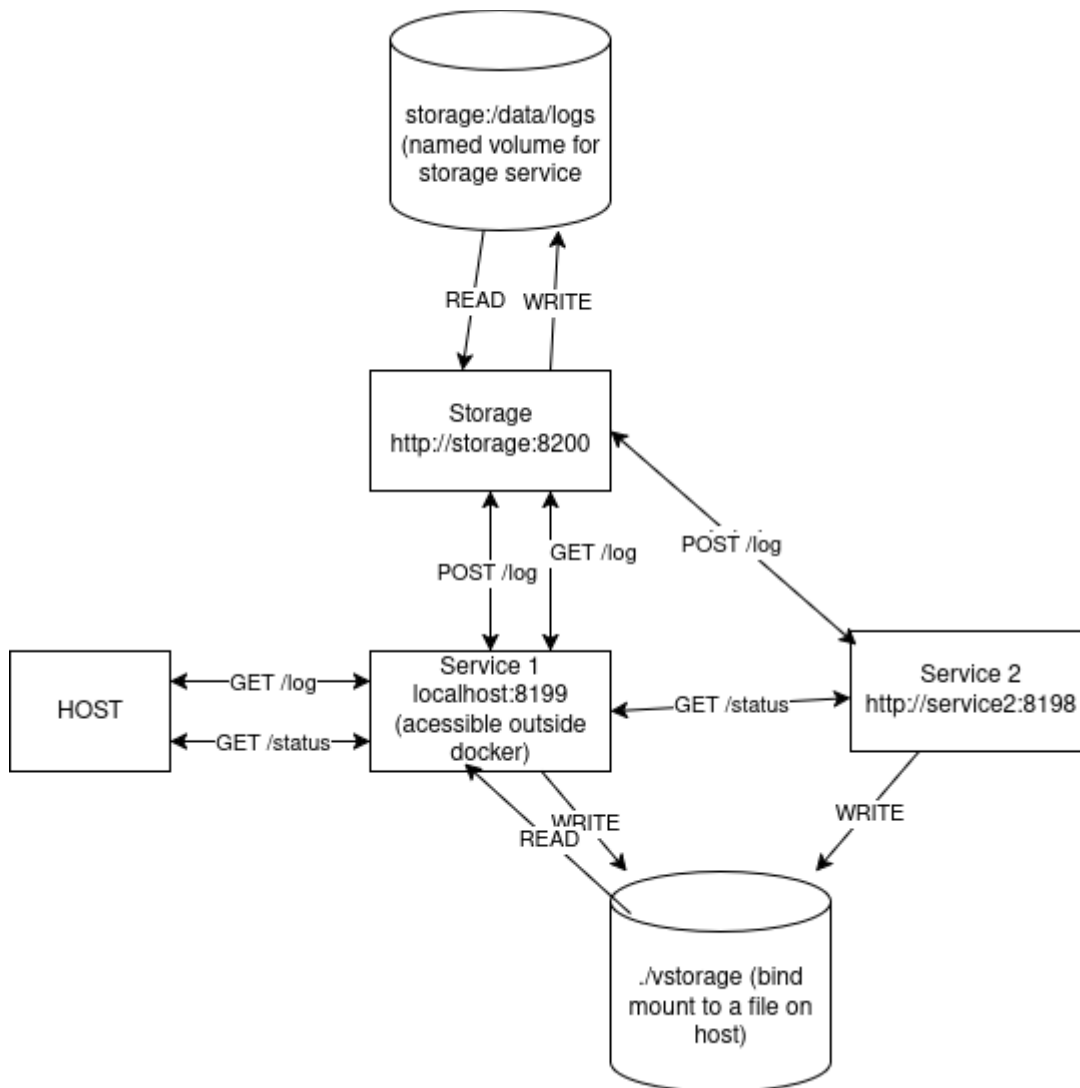
Desktop specifications are:

- Operating system: Microsoft Windows 10 Pro 64-bit
- Processor and architecture: AMD Ryzen 9 3900X (x86_64)
- Virtual machine: WSL2 with Ubuntu-22.04 LTS
- Docker version: 27.3.1, build ce12230
- Docker Compose version: v2.30.3-desktop.1

Laptop specifications are:

- Operating system: Linux Mint 22 Wilma
- Processor and architecture: Intel Core i5-7200U (x86_64)
- Docker version: 28.1.1 build 4eba377
- Docker Compose version: v2.35.1
- docker-compose version (mentioned in the assignment so tested that the compose worked with it): 1.25.0, docker-py version: 4.1.0, CPython version: 3.8.10

2. Diagram of services, networks, storage, with names and IP addresses



3. Status Records Analysis

The saved records include three pieces of information: a timestamp, uptime, and free disk space.

The timestamp is correct and useful. For example, Service1 produces "2025-09-14T12:06:18Z", which is an ISO-8601 UTC value. Using UTC formatted dates is a standard way to store datetime values without being dependent on time zone. This also makes it easy to compare records across services. Uptime is less relevant in its current form.

For uptime Service1 uses `Environment.TickCount64` and Service2 uses `sysinfo::System::uptime()`. Both report the host's uptime (e.g. "6 hours since last reboot"). In a production setup it doesn't give much valuable information because it does not give information about the container. If a container is restarted while the host keeps running, the record will misleadingly suggest the service has been up for 53 hours. A better solution would be to record the process or container uptime.

The disk space measurement in the status records (e.g. 173206 MBytes) reflects the free space of the host filesystem that Docker uses for the container's root (`/`). In this case, it means the disk space my

computer had left on that partition at the time of measurement. This is not very relevant information for a real containerized application.

4. Persistent Storage Solutions

4.1 mounted file on host (vStorage)

The bind-mounted file makes the contents immediately visible on the host and easy to inspect with commands like `cat ./vstorage`. However, this design tightly couples the container to the host's filesystem. The container needs the host file to exist beforehand, and for example file permissions may cause portability issues. It also breaks the principle of container isolation, since the container writes directly into the host's directory tree. As the file content is not managed by docker it can cause confusion as the file content stays even after cleaning up all containers and volumes. For these reasons this solution is fragile and not recommended to be used in production setups.

4.2 Named volume (storage service)

The storage service with named volume is more in line with the best practices. In this solution logs are written through an API into the storage container, and Docker manages the persistence in the named volume (`storage:/data/logs`). This isolates application code from the host filesystem and ensures the data remains available even if the storage container is stopped or rebuilt. It is also easier to share data consistently between services. The drawback of using docker managed volume is that the data is not immediately visible on the host.

5. Instructions to clear persistent storage.

- Ensure that container are down
- Run the following command: `docker volume rm devops-course_storage`

6. Challenges & Reflections

I think that the hardest thing in this exercise wasn't any single thing but managing the whole thing and getting every detail right. The assignment document wasn't the clearest and it required some time to grasp what is wanted. About the things that had to be done, the mounted file on the host caused the most issues for me because it was a thing that i had not done before. Also, using multiple different programming languages did bring its own challenges. I had used python and csharp quite a lot but had little experience with rust beforehand. Python and .net apps are also pretty straightforward to dockerize but rust package manager and optimizing compilation of the rust app required some thought. The programs themselves and networking were pretty simple. Overall I think I got some nice hands on experience with docker compose and especially volumes. I hadn't been using volumes too much before since most solutions that I have worked with have had separate non containerized databases.